Single Instruction Multiple Data – ARM NEON

Sorbonne Université – Master SESI – MU4IN112 – PPSE

Adrien Cassagne

March 26, 2025

Table of Contents

1 Vector and SIMD Programming Models

- Vector and SIMD Programming Models
- NEON SIMD Programming

Single Instruction Multiple Data (SIMD)

Example of scalar instructions:

```
loop:
  load a, [p_a]
  load b, [p_b]
  add c, a, b # c[i] = a[i] + b[i]
  store [p_c], c
  add p_a, p_a, 4 # 4 bytes, 32-bit
  add p_b, p_b, 4 # 4 bytes, 32-bit
  add p_c, p_c, 4 # 4 bytes, 32-bit
  add i, i, 1
jumple i, 10000, loop
```

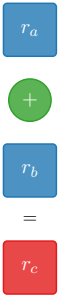
Example of SIMD instructions:

```
loop:
  vload va, [p_a] # read 4 elements
  vload vb, [p_b] # read 4 elements
  vadd vc, va, vb # add 4 elements
  vstore [p_c], vc # write 4 elements
  add p_a, p_a, 4*4 # 4*4 bytes, 128-bit
  add p_b, p_b, 4*4 # 4*4 bytes, 128-bit
  add p_c, p_c, 4*4 # 4*4 bytes, 128-bit
  add i, i, 4 # increment i of 4
jumple i, 10000, loop
```

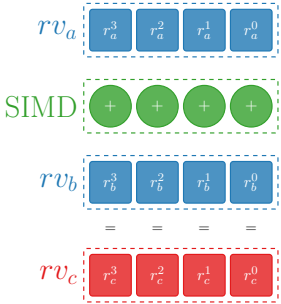
- The `vadd`, `vload` and `vstore` instructions perform operations on 4 elements at a time
  - 4 times fewer instructions (potential acceleration of 4)
  - The number of elements in a vector register = the # lanes (here 4)
  - Each element of `vc[i..i+4]` is computed in parallel

Single Instruction Multiple Data (SIMD)

- Scalar instruction: produces data in 1 cycle (to simplify)



- A SIMD instruction produces n data in 1 cycle (to simplify)



- SIMD instructions operate on so-called "vector" registers (rv_a, rv_b, rv_c)

SIMD and Vector Instruction Sets – Part 1

Date	ISA	Type	Micro-architecture	Registers Size
1997	MMX	SIMD	Intel Pentium	64-bit
1999	Altivec	SIMD	Apple+IBM PowerPC	128-bit
1999	SSE	SIMD	Intel Pentium III	128-bit
2000	SSE2	SIMD	Intel Pentium IV	128-bit
2004	SSE3	SIMD	Intel Pentium IV (Prescot)	128-bit
2005	NEONv1	SIMD	ARMv7	128-bit
2008	SSE4	SIMD	Intel Core 2 Duo	128-bit
2011	AVX	SIMD	Intel Sandy Bridge (Core i)	256-bit
2012	NEONv2	SIMD	ARMv8	128-bit
2013	AVX2	SIMD	Intel Haswell (Core iX)	256-bit
2016	AVX-512	SIMD	Intel Xeon Phi	512-bit
2017	SVE	Vector	ARMv8	–
2019	SVE2	Vector	ARMv9	–
2021	RVV	Vector	RISC-V	–

SIMD and Vector Instruction Sets – Part 2

Date	ISA	Type	Micro-architecture	Registers Size
1997	MMX	SIMD	Intel Pentium	64-bit
1999	Altivec	SIMD	Apple+IBM PowerPC	128-bit
1999	SSE	SIMD	Intel Pentium III	128-bit
2000	SSE2	SIMD	Intel Pentium IV	128-bit
2004	SSE3	SIMD	Intel Pentium IV (Prescot)	128-bit
2005	NEONv1	SIMD	ARMv7	128-bit
2008	SSE4	SIMD	Intel Core 2 Duo	128-bit
2011	AVX	SIMD	Intel Sandy Bridge (Core i)	256-bit
2012	NEONv2	SIMD	ARMv8	128-bit
2013	AVX2	SIMD	Intel Haswell (Core iX)	256-bit
2016	AVX-512	SIMD	Intel Xeon Phi	512-bit
2017	SVE	Vector	ARMv8	–
2019	SVE2	Vector	ARMv9	–
2021	RVV	Vector	RISC-V	–

- A large number of existing instruction sets
- Some are backward-compatible
  - AVX-512 → AVX → SSE → MMX
  - NEONv2 → NEONv1
- Most instruction sets are incompatible with each other
  - Each instruction set is written differently

Assembly Code

Example of single precision floating-point addition according to different ISAs:

x86 SSE

```
addps xmm, xmm
```

ARM NEON

```
FADD Vd.4S,Vn.4S,Vm.4S
```

x86 AVX

```
vaddps ymm, ymm, ymm
```

ARM SVE

```
FADDP Zresult.S, Pg.M, Zresult.S, Zop2.S
```

x86 AVX-512

```
vaddps zmm, k, zmm, zmm
```

RISC-V Vector extension

```
vadd.vv vd, vs2, vs1, vm
```

- As many different ways as there are instruction sets
- Assembly code is not portable :-(
- A major productivity brake

Intrinsic Functions: Example of Addition

x86 AVX

```
__m256 _mm256_add_ps (__m256 a, __m256 b);
```

x86 AVX-512

```
__m512 _mm512_mask_add_ps (__m512 src, __mmask16 k, __m512 a, __m512 b);
```

ARM NEON

```
float32x4_t vaddq_f32 (float32x4_t a, float32x4_t b);
```

ARM SVE

```
svfloat32_t svaddp_f32_x (svbool_t pg, svfloat32_t op1, svfloat32_t op2);
```

RISC-V Vector extension

```
vfloat32m1_t vfadd_vv_f32m1_m (vbool32_t mask, vfloat32m1_t maskedoff, vfloat32m1_t op1, vfloat32m1_t op2, size_t vl);
```

Intrinsic Functions

Definition: An intrinsic function is a function to which one (and only one) assembly instruction corresponds.

- As many different ways as there are instruction sets
- Code with intrinsic functions is not portable
- Better productivity than pure assembly programming
  - The compiler takes care of register allocation
  - Easy to interface with C and C++ code
  - Often the chosen method by embedded system developers

Work with the Compiler

- In some cases, the compiler can automatically vectorize the source code
  - Works when computations are regular and with `-O3` optimization level (GCC)
  - You can display a vectorization report to see which parts of the code have actually been vectorized
- The compiler often does not vectorize well
  - Either it does not vectorize at all
  - Or it vectorizes a little, but we could do much better
- It is possible to help the compiler with annotations (ex. OpenMP SIMD)
- In HPC, a lot of code uses the compiler + annotations
  - Supercomputers evolve fast
  - Relying on the compiler guarantees source code portability

Table of Contents

2 NEON SIMD Programming

- Vector and SIMD Programming Models
- NEON SIMD Programming

Source of Inspiration

This section of is strongly inspired by the excellent slides of Pr. Lionel Lacassagne (Sorbonne University).

Special thanks to him for sharing its work!

Strongly-typed Data Types

- C89 and before
  - { `signed`, `unsigned` } × { `char` }, { `short`, `long` } × { `int` }, `float`, `double`
  - ➔ `unsigned short`, `signed long int`, ...
  - Major portability problem: type size can change depending on arch. and OS
- From C99 (`stdint.h`)
  - Fixed-width integer types: `int8_t`, `int16_t`, `int32_t`, `int64_t`, `uint8_t`, `uint16_t`, `uint32_t`, `uint64_t`
  - But the fixed-width floating-point data types are missing
- ARM choose to use the same philosophy
  - `int8_t`, `int16_t`, `int32_t`, `int64_t`, `uint8_t`, `uint16_t`, `uint32_t`, `uint64_t`, `float16_t`, `float32_t`, `float64_t`
  - ➔ Fixed-width floating-point data types are present!

Strongly-typed SIMD Data Types – Part 1

SIMD types use strong typing: `<type><size>x<number of lanes>_t`

- 64-bit registers
  - Signed integers: `int8x8_t`, `int16x4_t`, `int32x2_t`, `int64_t`,
  - Unsigned integers: `uint8x8_t`, `uint16x4_t`, `uint32x2_t`, `uint64_t`,
  - Floating-point numbers: `float16x4_t`, `float32x2_t`, `float64_t`
  - Polynomial: `poly8x8_t`, `poly16x4_t`
- 128-bit registers
  - Signed integers: `int8x16_t`, `int16x8_t`, `int32x4_t`, `int64x2_t`,
  - Unsigned integers: `uint8x16_t`, `uint16x8_t`, `uint32x4_t`, `uint64x2_t`,
  - Floating-point numbers: `float16x8_t`, `float32x4_t`, `float64x2_t`
  - Polynomial: `poly8x16_t`, `poly16x8_t`

Strongly-typed SIMD Data Types – Part 2

SIMD types use strong typing: `<type><size>x<number of lanes>_t`

The availability of the different types and their associated instructions depends on the architecture version ({ v7, v8 } × { AArch32, AArch64 }), their specialization (ARM v8.2 for `float16_t`) and the implementations by designers (Apple, Nvidia, Samsung, Qualcomm, Huawei, ...).

Type of Instructions

4 main types of SIMD instructions:

1. Initialization and memory access
   - `load`, `store`, `set`
2. Arithmetic and logic
   - `add`, `mul`, `div`, `shift`, ...
   - `and`, `or`, `xor`, `not`
3. Comparison
   - $<$, $\leq$, $=$, $\neq$, $\geq$, $>$ (Fortran mnemonics: lt, le, eq, neq, ge, gt)
4. Shuffle & permutation
   - to reorganize values in a SIMD register, or in memory
   - because without it, some computations would be inefficient in SIMD

Type of Instructions

4 main types of SIMD instructions:

1. Initialization and memory access
   - `load`, `store`, `set`
2. Arithmetic and logic
   - `add`, `mul`, `div`, `shift`, ...
   - `and`, `or`, `xor`, `not`
3. Comparison
   - $<, \leq, =, \neq, \geq, >$ (Fortran mnemonics: lt, le, eq, neq, ge, gt)
4. Shuffle & permutation
   - to reorganize values in a SIMD register, or in memory
   - because without it, some computations would be inefficient in SIMD

It is not required to know all the intrinsics but we need to be able to find the instruction we are looking for from the technical documentation.

Initialization – vdup_n / vmov_n

- Set of values (or variables)
  - C++ style: `T x = {...};`
  - Example: `uint32x4_t x = {1, 2, 3, 4};` ➔ `x = [1, 2, 3, 4]`
  - Possible with scalar variables of compatible type (in this case `uint32_t`)
- Unique value (or unique variable) – `vdup_n_T`, `vdupq_n_T`
  - `uint32_t u = 1;`
  - Example 64-bit: `uint32x2_t x = vdup_n_u32(1);` ➔ `x = [1, 1]`
  - Example 128-bit: `uint32x4_t x = vdupq_n_u32(u);` ➔ `x = [1, 1, 1, 1]`
- Unique value (or unique variable) – `vmov_n_T`, `vmovq_n_T`
  - `uint32_t u = 1;`
  - Example 64-bit: `uint32x2_t x = vmov_n_u32(1);` ➔ `x = [1, 1]`
  - Example 128-bit: `uint32x4_t x = vmovq_n_u32(u);` ➔ `x = [1, 1, 1, 1]`

Access to a Lane – vset_lane / vget_lane

- Set – `dst = vset_lane_T(val, src, i)`
  - Copy `src` into `dst` and write `val` in the `i`-lane of `dst`
  - To modify a register within a loop (efficiently)
- Get – `val = vget_lane_T(src, i)`
  - Extract the `i`-element
- Example

```
uint32x4_t x = (uint32x4_t) {10, 11, 12, 13}; // x = [10 11 12 13]
uint32x4_t y = vsetq_lane_u32(14, x, 3);      // y = [10 11 12 14]
uint32_t   t = vgetq_lane_u32(y, 2);          // t = 12
```

Memory Access – vld1 / vst1

- LoaD – `vld1_T(addr)`, `vld1q_T(addr)`
- STore – `vst1_T(addr, x)`, `vst1q_T(addr, x)`

```
// 64-bit loads ('T8', 'T16' and 'T32' are data pointers, 'r<T>_64' are 64-bit registers)
uint8x8_t  r8_64  = vld1_u8 (T8 ); // [0, 1, 2, 3, 4, 5, 6, 7]M
uint16x4_t r16_64 = vld1_u16(T16); // [01, 23, 45, 67]M
uint32x2_t r32_64 = vld1_u32(T32); // [0123, 4567]M
// --------------------------
// 128-bit stores ('T8', 'T16' and 'T32' are data pointers, 'r<T>_128' are 128-bit registers)
vst1q_u8 (T8 , r8_128 ); // [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F]M
vst1q_u16(T16, r16_128); // [01, 23, 45, 67, 89, AB, CD, EF]M
vst1q_u32(T32, r32_128); // [01234567, 89ABCDEF]M
```

- The basic memory access instructions manipulate 1 set of elements, but there are also instructions that manipulate 2, 3 or even 4 sets of elements
  - For strided accesses 1:2, 1:3, 1:4
  - For conversions: Array-of-Struct - Struct-of-Array (AoS / SoA)

Display for Debug

- No working code without debugging!
- Here is an example of printing SIMD register values
  - 8-bit elements & 128-bit SIMD register ➔ 16 elements

```
1 void display_r8_128(uint8x16_t r8_128, char *format) {
2     uint8_t T[16];                // declare and allocate array in memory
3     vst1q_u8(T, r8_128);          // write the vector register in memory
4     for (int i = 0; i < 16; i++) // for each element of the SIMD register
5       printf(format, T[i]);       // print the value of lane i
6 }
```

- SIMD register elements (= lanes) displayed in ascending order
  - As if the data were in memory
  - Same display on little and big endian architectures

Binary Operations – `not` / `or` / `and` / `xor`

- Basic operators
  - NOT: `vmovn` (MOVe Not)
  - AND: `vand`
  - OR: `vorr` (OR Register)
  - XOR: `veor` (Exclusive OR)
- Advanced / specialized operators
  - OR NOT: `vorn` (OR Not)
  - AND NOT: `vbic` (BIt Clear)
  - REV: `vrbit` (Reverse BIT) inverts the bits of each element
  - XOR[2]: `veor3` *3-way exclusive OR*
  - `vbcax` *bit clear and exclusive OR*

Arithmetic Operations – `add` / `sub` / `mul` / `div`

- Classic arithmetic operations
  - ADD: `vadd` 8, 16, 32, 64
  - SUB: `vsub` 8, 16, 32, 64
  - NEG: `vneg` 8, 16, 32, 64
  - MUL: `vmul` 8, 16, 32, 64
  - DIV: `vdiv` 16, 32, 64 (only for floats)
- Specialized arithmetic operations
  - MUL: `vmul_lane(a, b, i)`: $a \times b_i$
  - MUL: `vmul_n(v, s)`: multiplication by a scalar value

Saturated Arithmetic Operations – Integers

1. General case of arithmetic: Size change (C and mathematics)
   - Addition $n + n \rightarrow n + 1$ bits so $2n$ bits
   - Multiplication: $n \times n \rightarrow 2n$ bits
   - $\Rightarrow$ Loss of parallelism ($\div 2$)
   - The size change prevents the compiler from vectorizing integer codes
   - Example `(a + b + c + d) / 4`
2. Special case #1: Without size change
   - Addition: $n + n \rightarrow n$ bits because info on missing addition holdback
   - Multiplication: $n \times n \rightarrow 2n$ because operands on n bits
   - Signal and image processing
   - Bad example `(a / 4 + b / 4 + c / 4 + d / 4)` (loss of precision)
3. Special case #2: With only a few holdbacks
   - Often the case with audio
   - Computations without loss of parallelism + saturation of results that exceed (internal holdback test)

Saturated Arithmetic Ops – vqadd / vqsub

- Only for addition (`vqadd`) and subtraction (`vqsub`)
  - Unsigned saturated arithmetic `vqadd_u`, `vqsub_u` $[0, 2^n - 1]$
  - Signed saturated arithmetic `vqadd_s`, `vqsub_s` $[-2^{n-1}, 2^{n-1} - 1]$

```
uint8x8_t u = (uint8x8_t) {1, 2, 3, 4, 5, 6, 7, 8};
                          // [  1   2   3   4   5   6   7   8]
u = vqadd_u8(u, u); // [  2   4   6   8  10  12  14  16]
u = vqadd_u8(u, u); // [  4   8  12  16  20  24  28  32]
u = vqadd_u8(u, u); // [  8  16  24  32  40  48  56  64]
u = vqadd_u8(u, u); // [ 16  32  48  64  80  96 112 128]
u = vqadd_u8(u, u); // [ 32  64  96 128 160 192 224 255] <- sat(128 + 128) = sat(256) = 255
u = vqadd_u8(u, u); // [ 64 128 192 255 255 255 255 255]
u = vqadd_u8(u, u); // [128 255 255 255 255 255 255 255]
u = vqadd_u8(u, u); // [255 255 255 255 255 255 255 255]
```

Ordering instructions – `min` / `max` – Part 1

- Two missing scalar instructions
  - In scalar mode, use a control structure: `if(a<b) m=a; else m=b`
  - The extremum is computed by subtraction and comparison of the sign of the result: `s=a-b; if(s<0) m=a; else m=b;`
  - On simple processors, the control structure generates a pipeline stalls
  - But on complex processors, there is a conditional `move` (= `cmove`)
- In SIMD
  - The extremum is computed without any control structure
  - SIMD version more is efficient

Ordering instructions – `min` / `max` – Part 2

Extraction of min and max scalars from 2 arrays

- Array version (v1) (most compact)

```
for (int i = 0; i < n; i++) {
  if (A[i] < B[i]) {
    Min[i] = A[i]; Max[i] = B[i];
  } else {
    Min[i] = B[i]; Max[i] = A[i];
  }
}
```

- Version with ternary conditions (v2)

```
for (int i = 0; i < n; i++) {
  uint8_t a = A[i];
  uint8_t b = B[i];
  uint8_t m = (a < b) ? a : b; // better without if
  uint8_t M = (a < b) ? b : a; // 'cmov' instr.
  Min[i] = m; Max[i] = M;
}
```

- SIMD version

```
for (int i = 0; i < n; i += 16) {
  uint8x16_t a = vld1q_u8(A + i);
  uint8x16_t b = vld1q_u8(&B[i]);
  uint8x16_t m = vminq_u8(a, b);
  uint8x16_t M = vmaxq_u8(a, b);
  vst1q_u8(Min + i, m);
  vst1q_u8(&Max[i], M);
}
```

— SIMD code is easy to write from the scalar version 2

Comparison instructions – `vc`

- SIMD intrinsic names are derived from Fortran mnemonics
  - For integers and floats
  - `vclt <` LT: Less Than
  - `vcle <=` LE: Less or Equal
  - `vceq ==` EQ: EQual
  - `vcge >=` GE: Greater or Equal
  - `vcgt >` GT: Greater Than
- Extension to unary (monoadic) operators
  - Avoids the use of a second register initialized to zero
  - For integers signed and floats
  - `vcltz` < 0
  - `vclez` ≤ 0
  - `vceqz` $= 0$
  - `vcgez` ≥ 0
  - `vcgtz` > 0

Web References

- Neon Intrinsics Reference
  https://developer.arm.com/architectures/instruction-sets/simd-isas/neon/intrinsics
- Armv8 Instruction Set Architecture: https://developer.arm.com/docs/ddi0596/c/simd-and-floating-point-instructions-alphabetic-order
- Coding for NEON
  - Part 1: Load and Stores https://community.arm.com/developer/ip-products/processors/b/processors-ip-blog/posts/coding-for-neon---part-1-load-and-stores
  - Part 2: Dealing With Leftovers https://community.arm.com/developer/ip-products/processors/b/processors-ip-blog/posts/coding-for-neon---part-2-dealing-with-leftovers
  - Part 3: Matrix Multiplication https://community.arm.com/developer/ip-products/processors/b/processors-ip-blog/posts/coding-for-neon---part-3-matrix-multiplication
  - Part 4: Shifting Left and Right https://community.arm.com/developer/ip-products/processors/b/processors-ip-blog/posts/coding-for-neon---part-4-shifting-left-and-right
  - Part 5: Rearranging Vectors https://community.arm.com/developer/ip-products/processors/b/processors-ip-blog/posts/coding-for-neon---part-5-rearranging-vectors

Q&A

Thank you for listening!
Do you have any questions?